

# 1. 选择:

(1) 如果调用C++流进行输入输出, 下面的叙述中正确的是

A 只能借助于流对象进行输入输出 ✓

B 只能进行格式化输入输出

C 只能借助于cin和cout进行输入输出 ✗

D 只能使用运算符 >> 和 << 进行输入输出

2) 要利用 C++ 流进行文件操作, 必须在程序中包含的头文件是。

A) iostream

B) fstream file

C) strstream

D) iomanip

✓ (3) 在C++中, cin是个

A) 类

B) 对象 标准输入流对象

C) 模板

D) 函数

4) 在语句cin>>data;中, cin是

A) C++的关键字 B) 类名

C) 对象名 D) 函数名

5) 下列关于类定义的说法中, 正确的是

A) 类定义中包括数据成员和函数成员的声明

B) 类成员的缺省访问权限是保护的 ✗ private

C) 数据成员必须被声明为私有的 ✗

D) 成员函数只能在类体外进行定义 ✗

6) 对于一个类定义, 下列叙述中错误的是

A 如果没有定义拷贝构造函数, 编译器将生成一个拷贝构造函数

B 如果没有定义缺省的构造函数, 编译器将 一定 生成一个缺省的构造函数

C 如果没有定义构造函数, 编译器将生成一个缺省的构造函数和一个拷贝构造函数

D 如果已经定义了构造函数和拷贝构造函数, 编译器不会生成任何构造函数

(7) 有如下程序:

```
#include <iostream>
```

```
using namespace std;
```

```
//class Toy{
```

```
public:
```

```
Toy(char* _n) { strcpy (name, _n); count++;}
```

```
~Toy(){ count--; }
```

```
char* GetName(){ return name; }
```

```

static int getCount(){ return count; }
private:
    char name[10];
    static int count;
};
int Toy::count=0;
int mail()
{
    Toy t1("Snoopy"),t2("Mickey"),t3("Barbie");
    cout<<t1.getCount()<<endl;
    return 0;
}

```

Handwritten notes:   
 - Red box around `static int getCount(){ return count; }`  
 - Red arrow from `static int count;` to `count` in `getCount()`  
 - Red circles around `mail()` and `count` in `Toy::count=0;`  
 - Red text: `count++ count++ count++` with numbers 1, 2, 3 above them.  
 - Red text: `3` and `2` near the end of the code block.

运行时的输出结果是

- A) 1
- B) 2
- C) 3** ✓
- D) 运行时出错

80 有如下程序

```

#include<iostream>
using namespace std;
class A {
public:
    A(int i):r1(i) {}
    void print() {cout<<'e'<<r1<<'-';}
    void print() const {cout<<'C'<<r1*r1<<'-';}
private:
    int r1;
};
int main(){
    A a1(2); const A a2(4);
    a1.print(); a2.print();
    return 0;
}

```

Handwritten notes:   
 - Red circle around `const A a2(4);`  
 - Red text: `e2-C16-` with an arrow pointing to the output of `a2.print()`  
 - Red text: `16` near the `r1*r1` in the `print() const` method.

运行时的输出结果是

- A) 运行时出错
- B) E2- C16-** `e2-C16-`
- C) C4- C16-
- D) E2- E4-

90 有如下程序:

```

#include<iostream>
using namespace std;
class Name{
    char name[20];
}

```

构造函数

public:

Name(){

strcpy(name, ""); cout<<'?';

}

Name(char \*fname){

strcpy(name, fname); cout<<'?';

}

};

int main(){

Name names[3]={Name("张三"), Name("李四")};

Return 0;

}

运行此程序输出符号'?'的个数是

A) 0

B) 1

C) 2

D) 3

10) 若有如下类声明

class My Class {

public:

MyClass () {cout<<1;

};

执行下列语句

MyClass a, b[2], \*p[2];

以后, 程序的输出结果是

A) 11 B) 111 C) 1111 D) 11111

(11) 要定义一个引用变量p, 使之引用类MyClass的一个对象, 正确的定义语句是

A) MyClass p=MyClass;

B) MyClass p=new MyClass;

C) MyClass &p=new MyClass;

D) MyClass a, &p=a;

(12) 若MyClass是一个类名, 且有如下语句序列

MyClass c1, \*c2;

MyClass \*c3=new MyClass;

MyClass &c4=c1;

上面的语句序列所定义的对象个数是

A) 1

B) 2

C) 3

D) 4

构造函数

无参构造

有参构造

最后默认形参

最后一个为默认参数, 仍输出?

\*p[2]是对象指针数组, 无对象

对象指针数组, 没有定义一个对象, 不调用构造函数

对象数组

定义一个新对象, 的空间然后返回地址, 别名被推

相当于定义一个, 然后把地址赋给c3

14) 有如下程序:

```
#include<iostream>
using namespace std;
class Sample{
public:
Sample(){}
~Sample(){cout<<"*";}
};
int main(){
Sample temp[2], *pTemp[2];
return 0;
}
```

执行这个程序输出星号 (\*) 的个数为 (B)。

- A) 1    B) 2    C) 3    D) 4

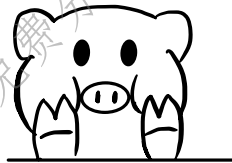
二次析构函数  
对象指针数组, 没有定义一个对象  
不调用构造函数

14) 有如下程序:

```
#include<iostream>
using namespace std;
class MyClass{
public:
MyClass(int i=0){cout<<1;}
MyClass(const MyClass&x){cout<<2;}
MyClass& operator=(const MyClass&x)
{cout<<3; return *this;}
~MyClass(){cout<<4;}
};
int main(){
MyClass obj1(1), obj2(2), obj3(obj1);
return 0;
}
```

运行时的输出结果是 (A)。

- A) 112444    B) 11114444  
C) 121444    D) 11314444



11 2 444

11 2 4 44

(15) 有以下类定义

```
class MyClass
{
public:
MyClass(){cout<<1;}
};
```

则执行语句 `MyClass a, b[2], *p[2];` 后, 程序的输出结果是

- A) 11    B) 111    C) 1111    D) 11111

对象指针数组, 不创建对象  
111

16) 有如下两个类定义



```

class AA{};
class BB{
    AA v1,*v2;
    BB v3;
    int *v4;
};

```

其中有一个成员变量的定义是错误的，这个变量是

- A) v1
- B) v2
- C) v3
- D) v4

类中不能有本身类型的变量  
类的对象一般放在类外  
或在主函数中定义，而不能放在类定义的范围内，但成员变量除外。  
的指针可以。

类BB没有定义完  
不能在类BB的定义中声明的对象

16) 有如下程序：

```

#include<iostream>
using namespace std;
class A{
public:
    static int a;
    void init () {a=1;}
    A (int a=2) {init (); a++;}
};
int A::a=0;
A obj;
int main ()
{
    cout<<obj.a;
    return 0;
}

```

构造

销毁

这个a是静态变量a  
这个a是形参  
内部名字覆盖外部。  
形参，不保存。  
运行完就消失。  
静态数据成员，外部定义。

运行时输出的结果是

- A) 0 B) 1 C) 2 D) 3

1

16) 有如下程序：

```

#include<iostream>
#include<iomanip>
using namespace std;
int main(){
    int s[]={123,234};
    cout<<right<<setfill('*')<<setw(6);
    for(int i=0; i<2; i++){ cout<<s[i]<<endl; }
    return 0;
}

```

运行时的输出结果是

- A) 123
- B) \*\*\*123
- C) \*\*\*123
- D) \*\*\*123

234

234

\*\*\*234

234\*\*\*

i=0 \*\*\* 123  
i=1 \*\*\* 234

16) 有如下类定义

```
class A {
```

```
    char *a;
```

```
public:
```

```
    A():a(0){}
```

```
    A(char *aa){ // 把aa所指字符串拷贝到a所指向的存储空间
```

```
        a= _____;
```

```
        strcpy(a,aa);
```

```
        strcpy(a,aa);
```

```
    }
```

```
    ~A(){delete []a;}
```

```
};
```

横线处应填写的表达式是

A) new char[strlen(aa)+1]

B) char[strlen(aa)+1]

C) char[strlen(aa)]

D) new char[sizeof(aa)-1]

26) 下列语句中错误的是 (A)。

A) const int a;

B) const int a=10;

C) const int\*point=0;

D) const int\*point=new int(10);

21) 下列程序段中包含4个函数，其中具有隐含this指针的是

```
int fun1();
```

```
class Test{
```

```
public:
```

```
    int fun2();
```

```
    friend int fun3();
```

```
    static int fun4();
```

```
};
```

A) fun1

B) fun2

C) fun3

D) fun4

(22) 在下面的运算符重载函数的原型中，错误的是

A) Volume operator - (double, double);

B) double Volume::operator- (double);

C) Volume Volume::operator- (Volume);

D) Volume operator - (Volume, Volume);

(23) C++ 流中重载的运算符 >> 是一个 ( )

cout <<  
cin >>

- A) 用于输出操作的非成员函数 B) 用于输入操作的非成员函数  
C) 用于输出操作的成员函数 D) 用于输入操作的成员函数

(24) 下列关于运算符重载的描述中，错误的是

- A) 可以通过运算符重载在C++中创建新的运算符 ~~✗ 不能重载而破创建~~  
B) 赋值运算符只能重载为成员函数 ~~✗ P136~~  
C) 运算符函数重载为类的成员函数时，第一操作数是该类对象 ✓  
D) 重载类型转换运算符时不需要声明返回类型 ✓ notice

重载类型转换运算符  
不需要声明返回类型

(25) 关于函数重载，下列叙述中错误的是

- A 重载函数的函数名必须相同 ✓  
B 重载函数必须在函数个数或类型上有所不同 ✓  
C 重载函数的返回值类型必须相同 ✗  
D 重载函数的函数体可以有所不同 ✓

(26) 下面程序中对一维坐标点类Point进行运算符重载

```
#include<iostream>
using namespace std;
class point {
public:
    point (int val) {x=val;} 构造函数 前增量 类内
    point& operator++ () {x++;return *this;}
    print operator++ (int) {point old=*this,++ (*this);return old;}
    int GetX () const {return x;} 有int 表示后增量
private:
    int x;
};
int main ()
{
    point a (10); 初始化 x=10
    cout<< (++a).GetX () ; 11
    cout<<a++.GetX () ; 11
    return () ;
}
```

编译和运行情况是

- A) 运行时输出1011  
B) 运行时输出1111  
C) 运行时输出1112  
D) 编译有错

(27) 若已经声明了函数原型“void fun(int a, double b=0.0);”，则下列重载函数声明中正确的是 (C)。

- A) void fun(int a=90, double b=0.0);  
B) int fun(int a, double B);  
C) void fun(double a, int B);

类型、个数  
完全一样

个数/类型或数据  
必须不一样

D) bool fun(int a, double b = 0.0);

28) 如果一个类的成员函数 print() 不修改类的数据成员值, 则应将其声明为

A. void print() const; B. const void print(); C. void const print(); D. void print(const);

不能改值, 则 const 成员函数

29) 在C++中, 关于下列设置参数默认的描述中, (C) 是正确的。

A. 不允许设置参数的默认值。

B. 设置参数默认值只能在定义函数时设置。

C. 设置参数默认值时, 应该是先设置右边的再设置左边的。

D. 设置参数默认值时, 应该全部参数都设置

30) f1(int) 是类A的公有成员函数, p是指向成员函数f1()的指针, 采用 (C) 是正确的。

A. p=f1

B. p=A::f1

C. p=A::f1()

D. p=f1()

## 2. 程序阅读:

(1) 有如下程序:

```
#include <iostream>
```

```
using namespace std;
```

```
class Machine{
```

```
static int num;
```

```
public:
```

```
Machine(){num++;}
```

```
static void showNum()
```

```
{cout<<num;
```

```
};
```

```
int Machine::num=0;
```

```
int main(){
```

```
Machine a[10], b;
```

```
Machine::showNum();
```

```
return 0;
```

```
} 运行这个程序的输出结果是 11。
```

(2) 下列程序的输出结果是 042。

```
#include <iostream>
```

```
using namespace std;
```

```
class Test {
```

```
public:
```

```
Test() {cnt++;}
```



```

~Test() {cnt--;} 析造
static int Count() { return cnt;}
private:
static int cnt;
};
int Test::cnt = 0;
int main()
{
    cout << Test::Count0 << ' ';
    Test t1, t2;
    Test* pT3 = new Test;
    Test* pT4 = new Test;
    cout << Test::Count0 << ' ';
    delete pT4;
    delete pT3;
    cout << Test::Count() << endl;
    return 0;
}

```

静态  
初始化

调用静态成员函数

0 4 2

(3) 下列程序的输出结果是 3, 3, 14。

```

#include <iostream>
using namespace std;
template<typename T>
T fun(T a, T b) { return (a <= b) ? b; }
int main()
{
    cout << fun(3, 6) << ' ' << fun(3.14F, 6.28F) << endl;
    return 0;
}

```

另一种写法

函数模板

若  $a \leq b$ , 则返回  $a$   $a \leq b ? a : b$

3, 3, 14

函数模板调用与普通函数调用无异。

(4) 给出下面程序的输出结果

```

#include <iostream>
using namespace std;
class base
{
    int x;
public:
    void setx(int a){x=a;}
    int getx(){return x;}
};
void main()
{
    int*p;
    base a;
    a.setx(15);
}

```

语法: 类名

base a; 无参

x=15

x=15

p = new int (a.get())

int \*p;

p = new int (a.get()); // 分配一个int空间的区域, get()为初始值  
cout << \*p;

15

(5) 给出下面程序的输出结果

```
#include <iostream>
using namespace std;
class base
{
private:
int x;
public:
void setx (int a){x=a;}
int getx () {return x;}
};
int main ()
{
base a,b;
a.setx (89);
b = a;
cout << a.getx () << endl;
cout << b.getx () << endl;
}
```

成员函数无多

调用时也要加()

89

89 89

6 有如下程序:

```
#include<iostream>
using namespace std;
class DA{
int k;
public:
DA (int x=1) : k (x) {}
~DA () {cout<<k;}
};
int main () {
DA d[] = {DA (3), DA (3), DA (3)};
DA *p = new DA[2];
delete[] p;
return 0;
}
```

未输入参数  
则默认为1

构造函数

它们的k=1

析构顺序相反

这个程序的输出结果是 11333。

7 有如下程序:

```
#include<iostream>
```

P176

执行析构函数, 输出 DA[] 的 11.  
再执行 a[] 的析构函数.  
输出 333

```

using namespace std;
class Wages{//“工资”类
double base;//基本工资
double bonus;//奖金
double tax;//税金
public:
    Wages(double CBase, double CBonus,
double CTax)
:base(CBase), bonus(CBonus),
tax(CTax){}
double getPay()const; // 返回应付工资额
Wages operator+(Wages w)const; // 重载加法
};
double Wages::getPay()const
{return base+bonus- tax;}
Wages Wages::operator+(Wages w)const
{return Wages(base+w.base, bonus+w.bonus,
tax+w.tax);
}
int main(){
Wages w1(2000,500,100),w2(5000,1000,300);
cout<<(w1+w2).getPay()<<endl;
return 0;
}
程序的输出结果是 8100

```

构造函数

重载

$7000 + 1500 - 400$

### 3. 程序填空:

1) 如下类定义中包含了构造函数和拷贝数的原型声明请在横线处写正确的内容，使拷贝构造函数的声明完整。

```
Class my Class{
```

```
Private:
```

```
Int data:
```

```
Public:
```

```
MyClass (int value) ;// 构造函数
```

```
MyClass (const myclass& another Object) ;// 拷贝构造函数
```

```
}
```

(const myclass &b)

多数据成员是类对象引用

2) 类Sample的构造函数将形参data赋值给数据成员data。请将类定义补充完整。

```
class Sample{
```

```
public:
```

```
Sample(int data=0);
```

```
Private:
```

```
Int data;
```

```
};
Sample::Sample(int data){
    this->data=data;
}
```

类内的对象

数据成员 data

this → data = data  
↓  
形参的 data,

形参

(3) 有如下程序, 请将横线处缺失部分补充完整。

```
#include<iostream>
using namespace std;
template<class T>
class Dataset{
    T *data;
    int size;
public:
    Dataset(T* arr, int length):size(length){
        data=new T[length];
        for(int i=0;i<length;i++){
            data[i]=arr[i];
        }
    };
    int main(){
        int arr[]={2,4,6,8,10};
        // 利用数组arr初始化类模板Dataset的对象set
        _____ DataSet<int> set(arr,5);
        return 0;
    }
```

类模板名

NO  
超后记加  
类模板

类模板对象创建

class name <类型> object;

就这样

如 <多> object;

对象名

创建类模板对象

arr

类模板名 <实际类型名> 对象名 (参数表);

(4) 已知如下程序得输出结果时23, 请将划线处缺失得部分补充完整。

```
#include<iostream>
using namespace std;
Class myclass{
    Public:
        Void print() cout<<23;
}
Int main(){
    myclass *p=new myclass();
    (*p).print();
    Return();
    Class sample{
    Public:
        Sample(){
            (*p)
        }
    }
```

用new 创建类对象空间

运算符优先级高于\*

构造函数

不理解

(5) 在答题纸上填上缺少的部分。源程序如下:



```

#include <iostream>
using namespace std;
class base
{
    int a,b;
    public:
    base( int x,int y){a=x;b=y;}
    void show ( const base &p,
    {
        cout<<p.a<<"", "<<p.b<<endl;
    }
}

void main()
{
    base b(78,87);
    b.show(b);
}

```

构造函数

或const base p或base p

base & p

(6) 在下面程序中的答题纸上填上适当的程序。使程序的输出结果如下： 67,90 源程序如下：

```

#include <iostream>
using namespace std;
class base
{
    private:
    int x,y;
    public:
    void initxy( int a,int b){x=a;y=b;}
    void show( base*p);
    inline void base::show ( base *p,
    {
        cout<< p- >x<<" "<<p- >y<<endl;
    }

    void print( base*p)
    {
        p -> show(p);
    }

    void main()
    {
        base a;
        a. initxy(67,90);
        print ( &a;
    }
}

```

=> x y

要用对象的地址

7) 请在下列程序中的空格处填写正确的语句:

```
class Sample{
public:
    Sample(){}
    ~Sample(){}
    void SetData(int data)
    { // 将 Sample 类成员变量 data 设置成形参的值
        Sample::data=data // 注意形参与成员同名
    }
```

```
private: int data;
};
```

8) 有如下类定义, 请将Sample类的拷贝构造函数补充完整。

```
class Sample{
public:
    Sample(){}
    ~Sample() {if(p) delete p;}
    Sample (const Sample& s){
        p=new int;*p=s.*p;
    }
    void SetData(int data) {p=new int(data);}
private:
    int*p;
};
```